



31. K-Fold Cross Validation

What if we move on from a single train-test split?

Previous Section:

 [30. Oversampling and Undersampling: Handling Imbalanced Data](#)

Contents:

[1. What is Cross-Validation?](#)

[2. K-Fold Cross Validation - The Overview](#)

[3. K-Fold Cross Validation with Python](#)

[4. Why Cross-Validation might be better than the classic train-validation-test split?](#)

[Summary](#)

[References](#)

There are many ways we can improve a machine learning model:

- We normalize data.
- We encode categories so machines can read them.
- We reduce dimensions to simplify the problem.
- We even fix the data itself—undersampling, oversampling, balancing the world we give to the model.
- But what if none of that is the real issue? What if the problem is not *how we train the model*. But *how we judge it (or how we split the data)*?

Think about this for a moment. You train a model. You test it once. You get an accuracy: 92% → Looks good, right?

But here's the uncomfortable question:

- What if that number is lying?
- What if your model just got lucky with that particular split?
- What if the test set was too easy?
- Or worse, what if it silently failed on data you didn't happen to include?

Because in Machine Learning, one split is just one perspective. And one perspective is never the full picture.

Imagine studying for an exam. You practice using only one mock test. You score high. You feel confident. But then the real exam comes, different questions. Slightly different patterns. And suddenly, everything falls apart. Was the model (or you), actually good? Or just good at *that one test*? This is where things start to shift. Instead of trusting a single split. We ask a better question:

■ ***"What if we test the model... multiple times?"***

Not once. Not twice. But across different views of the data. So that performance is not based on luck → But on consistency.

And that idea, that simple shift in thinking is what leads us to something much more reliable: **K-Fold Cross Validation**

1. What is Cross-Validation?

Now let's answer the question properly. What exactly is *Cross-Validation*? At its core, Cross-Validation is not about improving the model directly.

It's about something more subtle. It's about **measuring the model correctly**.

Because here's the truth that beginners often miss: A model is not judged by how well it performs on the data it has seen. But by how well it performs on data it has **never seen before**. And that idea? That's called **generalization**.

Cross-validation is a model evaluation technique used to assess how well a Machine Learning model generalizes to unseen data. Instead of relying on a single train-test split... We repeatedly train and evaluate the model on different subsets of the data. Not once. But many times.

And from that, we get something much more important than a single score → A **reliable and stable estimate of performance**.

But let's slow down and make this real. Imagine you are a student preparing for an exam. You have a book. You have practice questions. Now there are two ways you can study.

- **Strategy 1: The "One-Test Confidence":** You practice using only one mock test. You go through it carefully. You memorize patterns. You even recognize some repeated questions.

Then you take that same test again and score 95%. Feels amazing. But here's the problem: You didn't actually test your understanding. You tested your memory.

- **Strategy 2: The "Cross-Validation Mindset":** Now imagine a different approach. Instead of relying on one mock test. You split your practice into multiple smaller tests.

Each time: You study a portion of the material. You test yourself on a different portion, And you repeat this process, again and again.

Different questions. Different combinations. Different perspectives. Sometimes you score high. Sometimes you struggle. But over time, something interesting happens:

▮ You start to see your **true level**.

Not based on luck. Not based on familiarity. But based on **consistency**.

And that is exactly what Cross-Validation does. Instead of trusting a single train-test split. It creates multiple "mini-experiments" from the same dataset.

Each time: The model trains on one part of the data, and is tested on another part

Then we repeat the process. Over and over with different splits.

So what does this give us? It gives us something far more valuable than a single number:

- A performance that is **less sensitive to randomness**
- A model that is tested under **different scenarios**
- And most importantly, a much clearer picture of how the model will behave in the real world

Because in the end, Machine Learning is not about doing well once. It's about doing well **consistently**. And Cross-Validation is one of the simplest ways to

make sure of that.

In the next section, we'll take this idea one step further, and turn it into something structured and powerful: **K-Fold Cross Validation**.

2. K-Fold Cross Validation - The Overview

Let's take that idea of Cross-Validation and make it concrete. Because until now, everything sounds intuitive: "Test the model multiple times" → "Use different splits". But how exactly do we do that?

This is where **K-Fold Cross Validation** comes in. One of the most widely used—and most practical—techniques in Machine Learning.

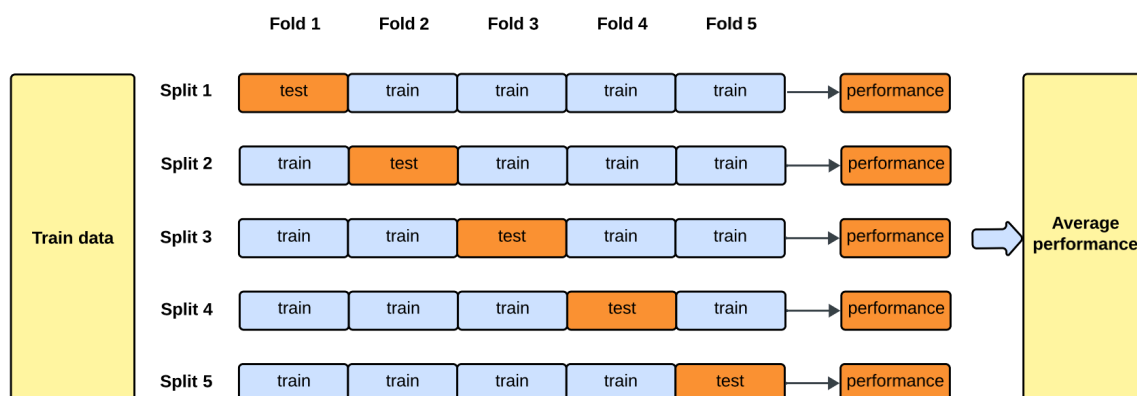
At its core, the idea is surprisingly simple:

- We split the dataset into k **equal parts** (called *folds*)
- We train the model on $k - 1$ **folds**
- And test it on the **remaining fold**

That's it. But the real power is not in one run... it's in repetition.

Instead of doing this once. We repeat the process k **times**. Each time, a different fold becomes the test set. So every part of the data gets a chance to be training data and testing data **exactly once**.

(The following image is used as reference only)



Let's slow down and visualize this: Imagine your dataset is like a stack of cards. Instead of cutting it once into "train" and "test". You divide it into multiple smaller stacks. And then you rotate them. Each stack takes turns being the "exam paper". While the others become the "study material".

Why does this matter?

Because in supervised learning, the goal is not just to perform well on training data. The real and ultimate goal is to **generalize**. To handle data the model has never seen before.

And here's the problem with a single train-test split: It might be **too easy**; It might be **too hard**; It might be **unlucky** → Especially when the dataset is small, imbalanced, or noisy → One split is just one story.

K-Fold Cross Validation fixes this. By making sure that:

- **Every data point is used for training** → **Every data point is also used for testing** → Just at different times.

The Process (Step-by-Step)

Let's say we choose: $k = 5$ (5-Fold Cross Validation)

And we have a dataset of **1000 samples**. So we split it into: 5 folds → Each fold contains 200 samples

Now the process begins.

- **Iteration 1:** Train on folds 2, 3, 4, 5 → (800 samples); Test on fold 1 → (200 samples)
- **Iteration 2:** Train on folds 1, 3, 4, 5 → Test on fold 2
- **Iteration 3:** Train on folds 1, 2, 4, 5 → Test on fold 3
- **Iteration 4:** Train on folds 1, 2, 3, 5 → Test on fold 4
- **Iteration 5:** Train on folds 1, 2, 3, 4 → Test on fold 5

Notice something important: Every fold is used as the test set **exactly once**. No data is wasted. No data is ignored.



Important Note (Don't Misunderstand This) - At this point, there's a very common mistake students make. They think that in each iteration, **we reshuffle and create new folds again** → **No. That is NOT what happens here.**

In **K-Fold Cross Validation**, the dataset is split into K **folds only once** (for example, 5 folds) → And then every iteration simply **reuses these same folds**. We are not creating new subsets each time. We are just **rotating which fold becomes the test set**.

Think of it like this: You have 5 fixed piles of data sitting on the table. You don't reshuffle them every time. → You just say:

- "This pile is the test set now"
- "The rest are for training"

Then next round, you pick another pile.

So remember: **Split once** → **Rotate many times**; **Not: Split again and again**

This detail might seem small, but misunderstanding it can lead to a completely wrong implementation.

With that clarification in mind, we move on to the final evaluation:

After all k iterations, we don't just keep one score. We now have k **different performance scores**: Accuracy; Precision; Recall; and F1-score. And instead of trusting one, we take the **average**. And that average? That becomes our final evaluation. Not perfect. But far more **stable**. Far more **trustworthy**.

Because now, the model has been tested on different subsets under different conditions and across multiple perspectives.

So in the end, K-Fold Cross Validation does something very important: It turns evaluation from a **single guess** into a **repeated experiment**. And in Machine Learning, that difference is everything.

3. K-Fold Cross Validation with Python

Let's bring K-Fold Cross Validation into Python (using the Scikit-Learn library).

```
import pandas as pd
from sklearn.datasets import load_breast_cancer

# Read the Breast Cancer
df = load_breast_cancer(as_frame=True)
X = df.data
y = df.target
print(f>Data shape: {X.shape}")
```

Data shape: (569, 30)

```
from sklearn.model_selection import KFold

# Define K-Fold Cross-Validation
# n_splits=5: splits the data into 5 folds
# shuffle=True: shuffles data before splitting to avoid bias
k_fold = KFold(n_splits=5, shuffle=True, random_state=42)
# Return the size of the training and testing fold
for train_index, test_index in k_fold.split(X):
    print(f"Train: {train_index.shape}; Test: {test_index.s
hape}")
```

Train: (455,); Test: (114,)

Train: (455,); Test: (114,)

Train: (455,); Test: (114,)

Train: (455,); Test: (114,)

Train: (456,); Test: (113,)

Why does the last fold have a different shape? Because 569 is not perfectly divisible by 5. Here: $569 \div 5 \approx 113.8 \rightarrow$ So what happens when we split into 5 folds? $\Rightarrow$ Most folds get 114 **samples**; One fold gets 113 **samples**. And that's completely fine.

K-Fold doesn't force perfect symmetry. It just keeps things as balanced as possible.

Also, since we have 5 folds (5 subsets) $\rightarrow$ We will perform 5 splits, in which in each split, a subset will be used as the testing set, while the rest are training sets.

```
from sklearn.tree import DecisionTreeClassifier

# Define the model
model = DecisionTreeClassifier(random_state=42)
```

Why don't we call `fit()` here? Because we are not training the model manually. We are handing everything over to **cross-validation**. The training will happen **inside the process**

In fact, something subtle happens behind the scenes: For each fold, a **fresh copy of the model** is created. That copy is trained on the training folds. Then evaluated on the test fold

So instead of a single model trained once, we actually have multiple models trained independently. Each one seeing a different version of the data.

```

from sklearn.model_selection import cross_validate

# Use cross_validate to calculate multiple metrics
scores = cross_validate(model, X, y, cv=k_fold, scoring=['accuracy', 'precision', 'recall', 'f1'])

# Access individual metrics (returns scores for each fold)
print(f"Test Accuracy: {scores['test_accuracy']}, Average: {scores['test_accuracy'].mean()}")
print(f"Test Precision: {scores['test_precision']}, Average: {scores['test_precision'].mean()}")
print(f"Test Recall: {scores['test_recall']}, Average: {scores['test_recall'].mean()}")
print(f"Test F1: {scores['test_f1']}, Average: {scores['test_f1'].mean()}")

```

Test Accuracy: [0.94736842 0.92982456 0.90350877 0.94736842 0.9380531], Average: 0.9332246545567457

Test Precision: [0.95774648 0.94805195 0.94117647 0.95774648 0.94117647], Average: 0.9491795693949795

Test Recall: [0.95774648 0.94805195 0.90140845 0.95774648 0.95522388], Average: 0.9440354474199335

Test F1: [0.95774648 0.94805195 0.92086331 0.95774648 0.94814815], Average: 0.9465112726598186

Each array represents **one fold**: 5 numbers → 5 different test results → 5 different “views” of the model → And then we take the **average**.

This is where everything comes together. Instead of saying: “My model accuracy is 94%”

We now say: *“Across multiple splits... my model consistently performs around 93%–94%”*

That statement is much more honest, much more reliable, and much more *real*

Because in Machine Learning, we don’t trust a model that performs well once. We trust a model that performs well, **again and again, under different conditions**.

4. Why Cross-Validation might be better than the classic train–

validation–test split?

Now let's compare two very different mindsets. On one side: The **classic split** (train–validation–test) → On the other: **Cross-Validation**

At first glance, both seem to do the same thing: Train the model, test it, evaluate it. But the difference is deeper than that. It's not just *how* we train. It's *how much we trust the result*.

(1) More Reliable Performance Estimates - With a single split, everything depends on **one decision**: How the data was divided, and that can be dangerous.

- A "lucky" split → The test set is easy → Performance looks amazing
- An "unlucky" split → The test set is hard → Performance drops

→ Same model. Different result.

Cross-Validation changes this completely. Instead of relying on one split, we evaluate the model across **multiple folds**. And then we average the results:

- Randomness is reduced → Variance is reduced → The estimate becomes **stable**

Think of it like this: One test score → could be luck; Multiple test scores → start to reveal the truth

(2) Better Use of Available Data - In a traditional split: Some data is **never used for training**. For example: 80% for training; 20% for testing. That 20% just sits there waiting to evaluate.

But in K-Fold Cross Validation, every data point gets two roles: Training data and Testing data. Just at different times.

This is especially important when the dataset is small, data is expensive, or especially when every sample matters → Because now, nothing is wasted.

(3) Reduced Risk of Overfitting to a Single Split - Here's something subtle, but very real.

When we tune a model using a fixed validation set. The model can slowly start to **adapt to that specific subset**. Not the problem. Not the data distribution. Just that slice.

Cross-Validation prevents this. Because the model is exposed to different training sets and different validation sets → Over and over again.

- So instead of learning: "How to do well on this split" → It learns: "How to do well in general"

(4) More Trustworthy Model Comparison - Imagine comparing two models: Model A with 92% accuracy and Model B with 90% accuracy. With a single split, can we really trust that difference? → Maybe Model A just got lucky.

With Cross-Validation: Both models are evaluated across the **same folds**. So the comparison becomes fair. Now when Model A consistently outperforms Model B, that difference actually means something.

A Clear Comparison

Aspect	Classic Train-Test Split	K-Fold Cross Validation
Data Splitting	One split (e.g., 80-20, 70-30)	Data split into K folds
Number of Evaluations	1	K evaluations
Test Set Usage	Fixed test set	Each fold becomes test set once
Training Data Usage	Only part of data	Almost all data used repeatedly
Stability of Results	Can vary a lot	More stable (averaged results)
Risk of Bias	High (depends on split)	Lower (multiple perspectives)
Model Comparison	Less reliable	More fair and consistent

The classic split is simple. Fast. And sometimes good enough.

But Cross-Validation is different. It doesn't just ask: "How well did the model perform?" → It asks something better:

■ **"How consistently does the model perform?"**

And in Machine Learning, consistency is what turns a good model into a **trustworthy one**.

Summary

K-Fold Cross Validation is powerful when you want **consistency over chance**. It moves us away from trusting a single split and toward evaluating a model across multiple perspectives.

Instead of asking: "Did the model perform well once?" → It asks: *"Does the model perform well... again and again?"*

By splitting the data into K folds and rotating the test set, we:

- Reduce the impact of randomness
- Use data more efficiently
- Get a more stable and reliable estimate of performance

And in the end, K-Fold Cross Validation reminds us of something important:

*"A good model is not the one that wins **one test**. It's the one that performs well... **no matter how the test changes**."*

References

<https://www.geeksforgeeks.org/machine-learning/k-fold-cross-validation-in-machine-learning/>

<https://machinelearningmastery.com/k-fold-cross-validation/>

<https://www.datacamp.com/tutorial/k-fold-cross-validation>

<https://www.youtube.com/watch?v=fSytzGwwBVw>

Next Section:

 [32. Gradient Descent Methods](#)